67250 IS MILIEUX

Phase B: Web Design, Development & Implementation

OBJECTIVE

Phase B builds your skills in designing and developing a fully functional, user-centered front-end website for MonoMuse, the museum whose web data you analyzed in Phase A.

You will translate your usability findings into intentional design, structural, and technical decisions. Your website must directly address the usability problem identified in Phase A, and every design choice — from information architecture to visual layout to interactive features — must clearly connect to your data findings and user insights.

Phase B moves you from analysis to execution. You will transform data insights into a fully functioning front-end, user-centered website for MonoMuse. By the end of this phase of the project, you will have:

- Apply user-centered design principles
- Structure content using semantic HTML
- Design layout and hierarchy using CSS
- Implement interactivity using JavaScript and the DOM
- Integrate media and at least one external API
- Apply accessibility best practices

This is an individual assignment. You may consult reference materials and seek guidance from your professor or TAs, but all submitted work must be your own.

BACKGROUND

MonoMuse is a contemporary museum that aims to provide an engaging, inclusive, and interactive experience for its visitors — both in person and online. While the museum offers compelling exhibitions and programming, its current website lacks clarity, intuitive navigation, and interactive features necessary to effectively support visitor planning and engagement.

In Phase A, you analyzed MonoMuse's web data to identify usability challenges and areas for improvement. Those findings now serve as the foundation for this redesign.

As a web designer and developer, your role is to create a modern, user-centered website that improves clarity, navigation, and engagement. The redesigned site should:

- Present exhibitions and museum content in a clear, organized way
- Make essential visitor information (hours, location, pricing, contact details) easy to find
- Support both first-time and returning visitors
- Incorporate interactive and dynamic features that enhance user experience
- Reflect a cohesive and intentional visual design

Your redesign should demonstrate a clear progression from research to implementation, showing how data insights inform information architecture, visual hierarchy, and technical functionality.

APPROACH

Phase B follows a structured, five-increment workflow that mirrors a real-world web design and development process. Although you do not submit each increment separately, you are expected to complete them alongside the corresponding class sessions. Staying on pace will help you make thoughtful design decisions, troubleshoot technical challenges early, and receive timely feedback.

Across these increments, you will translate your usability findings into a fully functioning, user-centered website by planning, designing, developing, and refining a multi-page front-end solution for MonoMuse.

- Increment 1: Museum Website Planning, Setup & HTML Foundations
- Increment 2: Usability, Wireframes Layout & CSS Visual Design
- Increment 3: Introduction to JavaScript
- Increment 4: Interactivity and Forms
- Increment 5: APIs, Media & Accessibility

REQUIREMENTS

Your MonoMuse website must demonstrate a clear connection between user needs, design decisions, and technical implementation. The following requirements define the minimum expectations for your project.

User Experience

Your website must be designed to support the following user stories at a minimum. These represent core visitor needs that your MonoMuse website is expected to address.

- **Museum Overview:** As a visitor, I want to understand what MonoMuse is and what it offers so that I can decide if I want to visit.
- **Exhibitions:** As a visitor, I want to view current exhibitions so that I can decide what I am most interested in seeing.
- **Visitor Planning (Hours & Ticket Pricing):** As a visitor, I want to quickly find hours of operation and ticket pricing so that I can plan my visit.
- **Directions & Location:** As a visitor, I want to find the museum's location and directions so that I can easily get there.
- **Ticket Purchase:** As a visitor (guest), student, or member, I want to buy a ticket online so that I can secure entry before arriving.
- **Membership or Donation:** As a visitor (guest), student, or member, I want to learn about membership or donation options so that I can support the museum.

Ticket Checkout Page

Your website must include a semi-functioning ticket checkout page that supports the "buy a ticket" user story. This page should simulate a realistic museum checkout experience (no real payment processing required). Your checkout page must include:

- Visit date selection (date picker) — required
- Ticket type selection (general, student, member) — required
- Ticket quantity selection (1–10, numeric only) — required

- Automatic price calculation that updates based on ticket type and quantity. For simplicity, assume all tickets cost $18 each (the price should automatically update based on quantity)
- Email — required; must validate format
- Zip code — optional, but if it is filled in, it must be a five-digit code
- Join our mailing list checkbox
- “Place Order” button that validates inputs and redirects to a confirmation page
- Confirmation page displaying total cost and clear order confirmation message. This is a simulated checkout. No payment processing is required
- Required fields must show clear error messages if missing/invalid
- Inputs must have clear labels and be easy to understand

Information Architecture and Structure

- Minimum 3 fully interlinked pages
- Clear and intuitive navigation
- Prominent visitor information (hours, pricing, location, contact)
- Proper heading hierarchy (h1–h6)
- Meaningful page titles
- At least one formatted HTML table
- Logical directory structure (root, views, static)

Visual Design and Layout

- Layout built using Flexbox or Grid
- Consistent color palette and typography system
- Clear visual hierarchy
- Intentional spacing and alignment
- Active navigation state
- Hover states for interactive elements
- No CSS frameworks or templates

Functionality and User Interaction

- Semi-functioning ticket checkout page
- Automatic price calculation
- Confirmation page with total
- At least one custom JavaScript DOM interaction
- At least one jQuery-based feature (accordion, animation, etc.)
- At least one API integration
- Interactive gallery or slideshow (at least 3 images)
- Embedded video

Accessibility and Professional Standards

- Descriptive alt text for all images
- Meaningful link text
- Keyboard accessibility
- No flashing content
- HTML passes W3C validation
- Organized and commented code
- External libraries cited in comments
- Proper media attribution (i.e., copyrights)

- No absolute file paths

SUBMISSION

Phase B requires both a **public GitHub repository** and a **PDF submission to Gradescope**.

1. Public GitHub Repository (Required)
You must deploy your project using GitHub.
Your repository must:

- Be **public**
- Contain all code for the project
- Include a clearly organized folder structure
- Contain separate subfolders for:
    - increment-1
    - increment-2
    - increment-3
    - increment-4
    - increment-5
    - final-project

Each increment folder should contain the work completed during that stage of development.
The final-project folder must contain your fully functioning, polished website.

All HTML, CSS, JavaScript, images, and supporting assets must be stored in the repository. No code should be submitted outside of GitHub.

Your final project must be fully deployable and viewable via GitHub Pages.

2. Gradescope PDF (Required)
You must submit a single PDF to Gradescope that includes:

- **Cover Page** with the title of the project, your name, Andrew ID, and the date
- The **MonoMuse Website**, submitted as:
    - A clickable link to your **public GitHub repository**
    - A clickable link to your **live deployed website (GitHub Pages).** The final project must function when accessed via GitHub Pages link.
- Your **Design Guide** (building off the template - colors, fonts, etc.)
- High-fidelity **Wireframes for each page** in your website
- A 2–3 page **Design Summary** explaining how your website addresses the usability problem identified in Phase A and how your design decisions connect to your data findings

The PDF should be cleanly organized and professionally formatted.

RUBRIC

Total: 70 Points

This project evaluates your ability to move from data-informed problem identification to a usable, accessible, and technically sound digital solution.

GitHub Repository & Deployment (3 pts)

- Public GitHub repository with required folder structure
- All increment folders and final project included
- Website successfully deployed via GitHub Pages

PDF Documentation Submission (5 pts)

- PDF includes GitHub repo link and deployed site link
- Design guide and high-fidelity wireframes included
- 2–3 page design summary explaining design decisions

Research-to-Design Connection (10 pts)

- Website clearly addresses the usability problem identified in Phase A
- Design, structure, and technical decisions reflect data findings and user insights
- Documentation explains how design choices solve the identified problem

User Experience & Required User Stories (10 pts)

- Website supports all required user stories
- Key information is easy to find
- Navigation supports both first-time and returning visitors

Ticket Checkout Experience (8 pts)

- Checkout page includes required inputs and validation
- Automatic price calculation based on ticket quantity
- Confirmation page displays total and order message
- Form labels and feedback are clear and usable

Information Architecture & Structure (8 pts)

- Minimum three fully interlinked pages
- Clear and intuitive navigation structure
- Proper page titles and heading hierarchy
- Logical directory structure (root, views, static)

Visual Design & Layout (8 pts)

- Layout implemented using Flexbox or Grid
- Consistent color palette and typography system
- Clear visual hierarchy and readable content layout
- Active navigation state and hover states for interactive elements

JavaScript & Interactivity (8 pts)

- At least one custom JavaScript interaction using the DOM
- At least one jQuery feature (accordion, animation, etc.)
- Interactive features enhance usability and user engagement

Media & API Integration (5 pts)

- Interactive image gallery or slideshow (minimum 3 images)

- At least one embedded video
- At least one external API integration (e.g., Google Maps)

Accessibility & Professional Standards (5 pts)

- Meaningful link text and keyboard-accessible navigation
- Descriptive alt text for images
- HTML passes W3C validation
- Code is organized, commented, and properly cited

Web Dev Incement 1

67250 IS MILIEUX

Phase B: Web Design, Development & Implementation
Increment 1: Museum Website Planning, Setup & HTML Foundations

---

Overview

In this increment, you will begin setting up the technical foundation for your MonoMuse website. The goal of this stage is **not to finalize your design**, but to establish the basic structure of your website and prepare your development environment.

You will create the initial repository, deploy your project to GitHub Pages, and build the core HTML structure of your site. This step focuses on organizing your project, creating the required pages, and implementing semantic HTML elements that will support your later design and functionality work.

Although visual styling and interactivity will be developed in later increments, the pages you create in this step should already reflect the **core user needs and navigation structure** identified in Phase A.

This increment focuses on **setup, structure, and content scaffolding** rather than visual design.

By the end of this increment, you will have:

- A GitHub repository created and deployed with GitHub Pages
- The required project directory structure established
- A multi-page HTML website with all core pages created
- HTML elements implemented for structure and content
- Basic navigation connecting all pages

After completing this increment, you will be able to:

- Set up and manage a GitHub repository for a web project
- Deploy and host a static website using GitHub Pages
- Build a structured multi-page website using HTML
- Use semantic HTML elements to organize page content
- Create basic tables and forms using HTML

Necessary Software

Google Chrome

Install Google Chrome if you do not already have it:
https://www.google.com/chrome/

You will view and test your website in Chrome only. This is the recommended browser for the course.

Visual Studio Code

Download and install VS Code:
https://code.visualstudio.com

This is the recommended editor for the course. Feel free to use other IDEs if you are more comfortable with them.

Recommended VS Code Extensions

- HTML CSS Support
- HTML Preview
- HTMLHint
- IntelliSense for CSS class names
- Live Server

Reference Resource

W3Schools is an excellent reference for HTML and front-end concepts:
https://www.w3schools.com/

Part 1: Directory Structure

Create the following folder structure in your workspace.

1. Create a folder named:

```
None
67250_technologyproject_phaseb
```

2. Inside it, create a folder named:

```
None
andrewid-increment1
```

(replace *andrewid* with your Andrew ID)

3. Inside andrewID-increment1, create:

- A views folder for HTML pages (except index.html)
- A static folder for images and other assets

```
None
andrewid-increment1/
│
├── views
├── static
```

Part 2: Create your first HTML file (the home page)

1. Open Visual Studio Code
2. Open the andrewid-increment1 folder
3. Create a new file named index.html
4. Use ! to generate the HTML skeleton

```
None
andrewid-increment1/
│
├── index.html
├── views
├── static
```

Part 3: Create content for index.html

1. Add the following to the <head> element:

```html
<meta name="author" content="Your name">
<meta name="keywords" content="Museum, Carnegie Mellon University, Information Systems">
<meta name="description" content="MonoMuse website redesign project">
<title>MonoMuse</title>
```

2. Add content to the <body> element. Include:

   A <header> containing:

   - An <h1> with the museum name MonoMuse
   - An empty <nav> element (more on this in the next part)

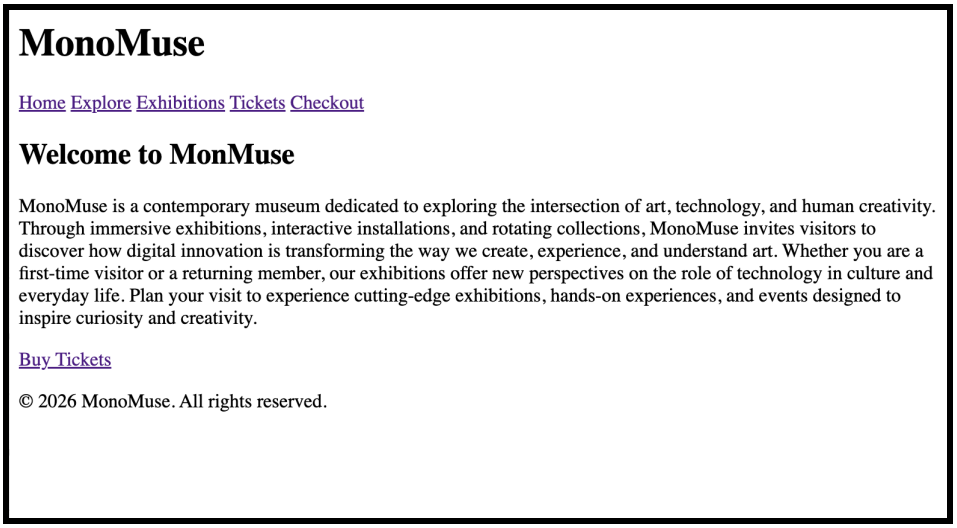
   A containing:

   - An <h2>: “Welcome to MonoMuse”
   - A short descriptive paragraph. For example:

     MonoMuse is a contemporary museum dedicated to exploring the intersection of art, technology, and human creativity. Through immersive exhibitions, interactive installations, and rotating collections, MonoMuse invites visitors to discover how digital innovation is transforming the way we create, experience, and understand art. Whether you are a first-time visitor or a returning member, our exhibitions offer new perspectives on the role of technology in culture and everyday life. Plan your visit to experience cutting-edge exhibitions, hands-on experiences, and events designed to inspire curiosity and creativity.

   - A Buy Tickets link or button (will eventually link to tickets.html)

   A <footer> with a copyright statement



Part 4: Multi-page structure and navigation

1. Inside the <header> <nav> add links to "Home", “Explore”, "Exhibitions", “Tickets" and “Checkout”. HINT: index.html will be in the root folder but the four other papers will be in the subfolder “views.”
2. Create the corresponding HTML files: “explore.html”, "exhibitions.html," “tickets.html” and “checkout.html” in the "views" subfolder by copying index.html and renaming it to the new file names. Update the paths in the <nav> so index.html is up a directory and the new pages are in the "views" subfolder.

Example:

```html
<nav>
  <a href="index.html">Home</a>
  <a href="views/explore.html">Explore</a>
  <a href="views/exhibitions.html">Exhibitions</a>
  <a href="views/tickets.html">Buy Tickets</a>
  <a href="views/checkout.html">Checkout</a>
  etc...
</nav>
```

Pathing Note:

- index.html is in the root folder

- All other pages are in the views folder
- The index.html file itself should link to index.html (or ./index.html).
- Pages inside the views/ folder should link back to the home page using ../index.html.
- Incorrect paths are a common cause of broken navigation. Verify all links work on every page.

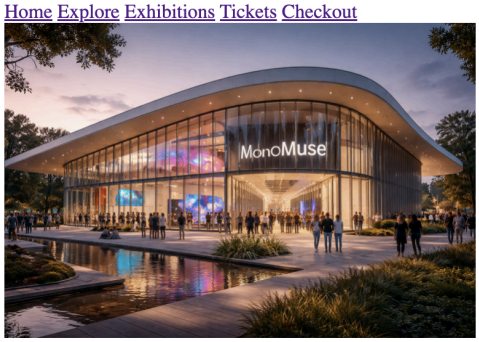
Part 5: Add additional content to each page

Explore Page (explore.html)

- `<h1>`: “Explore”
- `<img>` of an image of the museum with alternative text and sizing
- Content for (but not limited to - be creative) - must use the `<p>` and `<ul>` elements:
    - Mission statement
    - Who the museum serves
    - Founding year
    - Milestones

Explore

Home Explore Exhibitions Tickets Checkout



Mission Statement

MonoMuse is dedicated to exploring the evolving relationship between art, technology, and society. Our mission is to inspire curiosity and creativity by showcasing innovative exhibitions that highlight how digital tools, emerging technologies, and human imagination shape the future of artistic expression.

Milestones

- 2023: Expansion of the museum space and launch of international artist collaborations.
- 2018: Launch of the Interactive Media Lab, allowing visitors to experiment with creative coding.
- 2016: MonoMuse opens its doors with its first exhibition on digital storytelling.

© 2026 MonoMuse. All rights reserved.

Exhibitions Page (exhibitions.html)

- `<h1>`: “Exhibitions”
- At least two exhibitions, each with:
  - Title using `<h3>`
  - Short description using `<p>`

Exhibitions

Home Explore Exhibitions Tickets Checkout

Exhibition 1: Digital Dreams: Art in the Age of Algorithms

This exhibition explores how artists are using algorithms, artificial intelligence, and generative tools to create new forms of digital expression. Through interactive displays and dynamic visuals, visitors can see how machines and human creativity work together to produce art that evolves in real time.

Exhibition 2: Code & Canvas: The Language of Creative Technology

Code & Canvas highlights the ways programming has become a medium for artistic creation. Featuring works by digital artists and designers, the exhibition showcases installations, projections, and interactive pieces that reveal how lines of code can transform into immersive visual experiences.

© 2026 MonoMuse. All rights reserved.

Tickets Page (tickets.html)

- `<h1>`: “Buy Tickets”
- A `<table>` with columns:
  - Ticket Type (General, Student, Member)
  - Description
  - Price

Tickets

Home Explore Exhibitions Tickets Checkout

Ticket Type	Description	$Price$
Adult	General admission for ages 18 and over. Under 18 free of charge.	$18
Student	Student ID required	$10
Member	Member ID required	$15

© 2026 MonoMuse. All rights reserved.

Checkout Page (checkout.html)

- `<h1>`: “Checkout”
- A `<form>` with required fields:
  - Name (as text)
  - Email (as an email)
  - Ticket Quantity (as a number)
- A submit button
  (Form does not need to function yet)

Checkout

Home Explore Exhibitions Tickets Checkout

Name: (required)

Email: (required)

Ticket Quantity: (required)

Submit

© 2026 MonoMuse. All rights reserved.

Part 7: GitHub Setup and Deployment

Git is a version control system that manages and keeps track of your code. GitHub is a service that lets you host, share, and manage your code files on the Internet. GitHub uses Git underneath, and lets you manage your Git repositories or folders easily on its platform.

You will use a single GitHub repository for all 5 web development increments and the final project. You will create a new folder for each increment inside that repository.

Step 1: Create Your Repository (One Time Only)

1. Create a GitHub account (or recall your username/password)
2. Create a public GitHub repository named: 67250_techproject_phaseb
3. You only do this once for the entire semester.

Step 2: Add Your increment 1 Folder to GitHub (You will repeat this process at the end of each increment)

1. Select your github repository named 67250_techproject_phaseb
2. In the blue section you see on your screen click "upload an existing file" and drag and drop the contents of your andrewID-increment1 folder. Drag the full folder so it is in the root of your repo.
3. After all your files upload, click Commit changes (green button at bottom).

Optional: VS Code will also integrate with GitHub if you'd like to use those features instead.

Step 3: GitHub Pages Deployment

1. In GitHub, go to Repository → Settings → Pages
2. Select:
    - Branch: main
    - Folder: / (root)
3. Click Save

Your site will deploy from the root of your repository.

Note: Since increment 1 lives inside andrewID-increment1, you may need to manually navigate to that folder in the deployed URL. For example, the URL will be something like: https://jeriaq.github.io/67-250_techproject_phaseb/andrewid-increment1/index.html

Important notes

- You can create a landing page (index.html) that sits in your root
- You do NOT need to create a new repository for each increment.
- You do NOT need to create new branches for each increment.

Part 8: Submission

You do not need to upload anything to Canvas.

To receive credit:

- Your GitHub repository must be public
- The andrewID-increment1 folder must exist
- Files must be properly structured and accessible

TAs will review your GitHub repository directly.

🎉 Congratulations on completing Web Development increment 1! 👏

Web Dev Incement 2

67250 IS MILIEUX

Phase B: Web Design, Development & Implementation
Increment 2: Usability, Wireframes Layout & CSS Visual Design

---

Overview

This increment introduces CSS selectors, colors, fonts, the box model, display, flexbox, and responsive web design using media queries. You must meet every requirement outlined below, but you are encouraged to be creative with your visual design as long as the museum website remains easy to navigate—especially for Buy Tickets.

Your finished increment should show visible styling improvements across all pages and should work on both desktop and mobile screens.

By the end of this increment, you will be able to:

- Use CSS to style a website, including selectors, colors, fonts, and the box model to improve the visual design of pages.
- Apply layout and design techniques such as navigation styling, tables, forms, borders, padding, and hover effects.
- Integrate external resources, including linking a CSS file and using Google Fonts to improve typography.
- Create responsive designs using CSS concepts like display, flexbox, and media queries so the site works on different screen sizes.
- Improve user experience and navigation, especially by making important actions like Buy Tickets more visible and easy to find.

Part 1: Web Design Materials

As described in class, you should consider your information architecture and then create high-fidelity wireframes and a design guide for your project. You will submit these with your final project.

Part 2: Directory Structure and File Setup

1. Inside your `67250_techproject_phaseb` folder, create a new folder named `andrewid-increment2`
2. Copy the contents of your entire increment 1 folder into this new folder
3. Confirm you have the following structure:

```
andrewID-increment2/
|
├── index.html
├── views/
|   ├── checkout.html
|   ├── exhibitions.html
|   ├── explore.html
|   └── etc...
|
├── static/
|   └── (images, etc.)
```

4. Inside the `static` folder, create a new file named `style.css`
5. Link `style.css` in the <head> of **index.html** (root)

```html
<link rel="stylesheet" href="./static/style.css">
```

6. Link the CSS file in the <head> of **every HTML file in the views folder**:

```html
<link rel="stylesheet" href="../static/style.css">
```

Part 3: Basic Global Styling

In `style.css`, apply global styles using the universal selector `*`:

- Set font family to Arial

- Set padding to 5px
- Set margin to 5px
- Change the background color and text color of the website

Example:

```
CSS
* {
  padding: 5px;
  margin: 5px;
  background: #0b0b0c;
  color:  #f5f5f7;
}
```

Tip: Applying margin and padding using the universal selector (*) affects all elements (including navigation, tables, and forms). If your layout appears overly spaced or misaligned, consider scoping margin and padding to specific containers (e.g., body, .main, nav a).

Part 4: Basic Global Styling

1. Navigate to Google Fonts and search for **Roboto**
2. Click on the Roboto font and then **Get Font** → **Get embed code**
3. Copy the `<link>` tag into the `<head>` of every HTML file
4. In `style.css`, replace the font-family you had as Arial in `*` with Roboto
5. Do not copy the css code from Google Fonts. Instead, in your universal selector add:
    - font-optical-sizing: auto (Lets the browser automatically adjust the font's design for the size it's displayed at so it looks clearer and more balanced)
    - font-weight: 100 (controls how bold or thin the text is)
    - Line-height: 1.5
    - font-style: normal (optional - controls whether the text is italic or normal)

Example:

```
CSS
* {
  padding: 5px;
  margin: 5px;
  font-family: Roboto, Helvetica, Arial, sans-serif;
```

```
  font-optical-sizing: auto;
  font-weight: 100;
  line-height: 1.5;
}
```

Part 5: Header Style (Museum Branding)

1. In **every HTML file**, delete your page title `<h1>` element
2. Create a MonoMuse logo or download the example logo
3. Add the logo in your static folder for increment 2
4. Add the logo as an <img> with an id of “logo” to the top of the webpage just below the <header> element

Example:

```HTML
<img id="logo" src="./static/monomuselogo.png" alt="MonoMuse
Museum Logo">
```

5. In `style.css`, adjust the size and placement of `#logo`:

```CSS
#logo {
  width: 18%;
  display: block;
}
```

Part 6: Navigation Styling (Improve Buy Ticket Discoverability)

Your navigation should be clear and readable across all pages.

1. In `style.css`, select the `<a>` elements inside your nav and apply:

- Remove underlines (`text-decoration: none;`)
- Set link color
- Increase the font-weight
- Adjust the padding
- Creates a 1-pixel border that you can't see
- Rounds the corners of the border

Example:

```
CSS
nav a {
  text-decoration: none;
  color: #f5f5f7;
  font-weight: 400;
  padding: 8px 10px;
  border: 1px solid transparent;
  border-radius: 12px;
}
```

2. Because the navigation links already have a border (even though it is transparent), we can change that border to provide visual feedback when the user **hovers** over a link. Since the border is already part of the layout, changing its color on hover will **not cause the elements to shift or move**.
3. In `style.css`, add a **`:hover` pseudo-class** to the `<a>` element to modify the border when the user hovers over it.

Example:

```
CSS
nav a:hover {
  border-color: #4d5c3b;
  }
```

4. Add a CSS rule for the element with the ID `navselected` so that the text color changes to `#92ad70`. This style will highlight the navigation item representing the **current page**, helping users see which page they are on.
5. **Note** you will need to put the id="navselected" in the correct <a> for **each HTML** page.

Example:

```html
HTML
        <nav>
             <a id="navselected" href="index.html">Home</a>
             <a href="views/explore.html">Explore</a>
             <a href="views/exhibitions.html">Exhibitions</a>
             <a href="views/tickets.html">Tickets</a>
             <a href="views/checkout.html">Checkout</a>
         </nav>
```

```css
CSS
#navselected {
  color:#92ad70;
}
```

Part 7: Buy Ticket CTA Button Styling (Optional)

Recommendations:

- Make the **Buy Tickets** link (in the navigation or on the home page, or both) visually stand out (example options: bold, different color, background, border, etc.). Your goal is to reduce the “hard to find” problem identified in the database analysis.
- Add a class or id, such as class="cta", to the Buy Tickets link on your home page if you want to style independently from other links.

Select your Buy Tickets link on the home page and style it to look like a ‘call to action’ CTA button:

- background color
- no border
- rounded corners
- padding

This is optional but strongly recommended because it directly supports the project goal.

Part 8: Footer Styling

Your footer should be clear and readable across all pages.

6. In `style.css`, select the `<footer>` element and apply:
    - Add a border to the top of the footer (`border-top: 1px`)
    - Change the top border to a solid line with a color of your choice
    - Adjust the padding
    - Align the text to the center
    - Change the color of the text to your choice

Example:

```
CSS
footer {
  border-top: 1px solid rgba(245,245,247,.12);
  padding: 18px 16px;
  text-align: center;
  color: rgba(245,245,247,.72);
  }
```

Part 9: Table Styling (Tickets Page)

On your **tickets page**, style the ticket table.

1. Style the `<th>`:
    - padding: 12px top and bottom
    - background color and text color
    - width: fit-content
2. Add zebra striping using `nth-child()` on the rows.

Example:

```
CSS
th {
  padding-top: 12px;
  padding-bottom: 12px;
  width: fit-content;
}

tr:nth-child(even) {
  background-color: #f2f2f2;
```

```
}
```

Part 10: Form Styling (Checkout Page)

Style the checkout form so it is readable and user-friendly.

1. Select all form elements using a combination selector and:
    - Center elements with `margin: auto;`
    - Center text with `text-align: center;`

Example:

```CSS
form * {
  margin: auto;
  text-align: center;
}
```

Part 11: Browser Check and Validation

1. Open `index.html` and confirm styles apply
2. Confirm the Buy Tickets link stands out
3. Open your other HTML files and confirm styles apply
4. Confirm the table and form are styled correctly
5. Check your HTML code by using W3C Validator: https://validator.w3.org/
6. Fix major errors

Part 12: GitHub and Deployment

Add Increment 2 to GitHub

1. Select your github repository named 67250_techproject_phaseb
2. In the blue section you see on your screen click “upload an existing file” and drag and drop your andrewID-increment2 folder. Drag the full folder so it is in the root of your repo.
3. After all your files upload, click Commit changes (green button at bottom).

Optional: VS Code will also integrate with GitHub if you'd like to use those features instead.

Deploy on GitHub Pages

1. In GitHub, go to Repository → Settings → Pages
2. Select:
    - Branch: main
    - Folder: / (root)
3. Click Save

Your site will deploy from the root of your repository.

Note: Since Increment 2 lives inside andrewID-increment2, you may need to manually navigate to that folder in the deployed URL. For example, the URL will be something like: https://jeriaq.github.io/67-250_techproject_phaseb/andrewid-increment2/index.html

Important Notes

- You can create a landing page (index.html) that sits in your root
- You do NOT need to create a new repository for each increment.
- You do NOT need to create new branches for each increment.

Part 13: Submission

You do not need to upload anything to Canvas.

To receive credit:

- Your GitHub repository must be public
- The andrewID-increment2 folder must exist
- Files must be properly structured and accessible

TAs will review your GitHub repository directly.

🎉 Congratulations on completing Web Development Increment 2! 👏

Web Dev Incement 3

67250 IS MILIEUX

Phase B: Web Design, Development & Implementation
Increment 3: Foundations of JavaScript

Overview

This increment focuses on integrating JavaScript into your museum website to introduce fundamental programming concepts and practice DOM manipulation.

By the end of this increment, you will have:

- A personalized, time-based greeting on the homepage.
- The current year automatically updated in the footer.
- A button interaction (e.g., a feature to reveal ticket purchasing options).

By the end of this increment, you will be able to:

- Link JavaScript to your existing HTML files.
- Implement basic programming structures, including variables, functions, conditional statements, and loops.
- Update your website dynamically using event-driven interactions.
- Verify your code is functioning correctly by using browser alerts and the Chrome console

Part 1: Directory Structure and File Setup

To begin Increment 3, you must have successfully completed Increment 2. You should also have a completed design guide and low-fidelity wireframes that demonstrate clear attention to the information architecture, which you are now using to develop your high-fidelity wireframes.

1. Inside your `67250_technologyproject_phaseb` folder, create a new folder named:

```
None
andrewID-increment3
```

2. Copy the contents of your entire increment 2 folder into this new folder
3. Confirm you have the following structure:

```
None
andrewID-increment3/
│
├── index.html
├── views/
│   ├── checkout.html
│   ├── exhibitions.html
│   ├── explore.html
│   └── etc...
│
├── static/
│   └── (images, etc.)
│   └── style.css
```

4. Inside the `static` folder, create a new file named `script.js`
5. Link `script.js` in the `<head>` of index.html (root folder) including the `defer` attribute that ensures the script runs *after* the HTML loads:

```
HTML
<script src="./static/script.js" defer></script>
```

6. Link the JavaScript file in the `<head>` of every HTML file in the views folder:

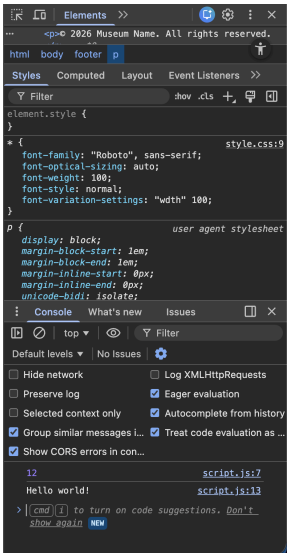
```
HTML
<script src="../static/script.js" defer></script>
```

Part 2: JavaScript Basics (Console Practice)

Variable Declarations and Console Output

1. Open `static/script.js` in VS Code and write the code needed to meet the requirements
2. Add a numeric addition:
    - Declare variables `var x = 5` and `var y = 7`

    - Create `var z = x + y`
    - Print `z` to the console using `console.log()`
3. Add a string concatenation:
    - Create string variables `A = "Hello "` and `B = "world!"`
    - Create `var C = A + B`
    - Print `C` to the console log, using `console.log()`
4. Verify your code is working correctly:
    - Open Chrome → View → Developer → JavaScript Console
    - Console should show:
        - `12`
        - `Hello world!`
    - Example:



Create a Basic Function

5. Define a function using VS code shortcut:

```
JavaScript
function name(params) {

}
```

6. Define a function named `sumnPrint` that accepts two parameters, `x1` and `x2`, as in `sumnPrint(x1, x2)`. The body of the function should perform the following actions:
    - Combine `x1` and `x2` (either by summing them if numeric or concatenating them if strings)

    - Output the resulting value to the console
7. Call it twice by typing:
    - `SumnPrint(x, y);`
    - `SumnPrint(A, B);`
8. Verify your code is working correctly. Console should show:
    - `12`
    - `Hello world!`
    - `12`
    - `Hello world!`

Add a Conditional Statement

9. Write an if/else statement that compares `C.length` to `z`
10. Use the VS code shortcut for ifelse (type the word if and select ifelse from the drop down menu in the frame):

```javascript
if (condition) {
    // Nested condition check
    if (condition) {
        // Code block for nested true
    }
    // End of nested check block
} else {
    // Code block for initial condition false
}
```

11. In the first condition brackets write the first conditional statement that checks if **`C.length > z`**
12. Followed by the rule statement in the curly brackets:
    - If this is true, log **`C`** to the console (if **`C.length`** is greater than **`z`**)
13. Then in the second condition brackets write the second conditional statement that checks if **`C.length < z`**
    - If this condition is true, log **`z`** to the console (if **`C.length`** is smaller than **`z`**)
14. In the last curly brackets state the else rule that applies in case none of the conditions apply.
    - Print `"good job!"` if they are equal
15. Verify your code is working correctly. The console should show `"good job!"`.

Arrays + Loops (Alerts)

1. Create two arrays:

```javascript
L1 = ["Watermelon","Pineapple","Pear","Banana"];
L2 = ["Apple","Banana","Kiwi","Orange"];
```

2. Write a function `findTheBanana()` that
   - Loops through the array (using a for loop)
   - Uses `alert()` when `"Banana"` is found (using an if statement)
3. Call the function with both arrays. More on: **forEach** JavaScript method.
4. Verify your code is working correctly. You should see only one alert for each array.
5. Now rewrite the function using `forEach()` and confirm it works again.
6. Once confirmed, feel free to comment these functions out.

Part 3: Time-Based Greeting (DOM Manipulation)

Add an ID to the Home Page

1. Now you will add a dynamic time based greeting to your home page. In **index.html**, add an `<h2>` element with an id=greeting somewhere near the top of your page *OR* edit the `<h2>` element you may already have:

```html
<h2 id="greeting">Welcome to the MonoMuse Museum</h2>
```

Define the Function greeting()

2. In `script.js`, declare a new variable called now, and assign to it a new time object (by using new Date())
3. Declare a new variable called hour and assign to it the extracted hour form now object by using the method .getHour
4. Define a function `greeting(x)` that:
   - Receives the hour as parameter,
   - Changes the inner html of `<h2> #greeting` based on the following condition:
     - Good morning (hour < 12)
     - Good afternoon (12 ≤ hour < 18)
     - Good evening (18 ≤ hour < 20)

- Good night (20 ≤ hour < 24 OR hour < 5)

Important: Implement your conditions in this order to avoid overlapping cases:

- Good night (h < 5 OR h >= 20)
- else Good morning (h < 12)
- else Good afternoon (h < 18)
- else Good evening

5. Verify your code is working correctly. Refresh `index.html` and confirm the greeting appears with the correct timing.

Fix the Error on the Other Pages (Required Debugging Task)

6. The greeting DOM code may create errors on pages that do not contain the `id="greeting"`. That is because our script.js is running on all pages.
7. Fix this by making sure your greeting code only runs when the element exists (or only on index.html).
   a. Hint for the first fix: Before updating an element, check that it exists on the page, for example, by confirming document.getElementById(...) returns a value.
   b. Hint for the second fix: You can use window.location to create this fix. (See https://www.w3schools.com/js/js_window_location.asp for more information).
8. Verify your code is working correctly:
   - Greeting shows on index.html
   - No console errors on any other pages

Part 4: Dynamic Footer Year(All Pages)

Update Footers

1. On every HTML page, add a paragraph inside the `<footer>`:

```html
<p id="copyYear"></p>
```

2. Remove the year from your footer text so the year is generated by JavaScript

Write addYear()

3. In `script.js`, write a function `addYear()` that:

- Uses `new Date().getFullYear()`
- Updates the `copyYear` paragraph

Call addYear() When the Page Loads

4. In the `<body>` tag of every html page, add:

```html
onload="addYear()"
```

5. Verify your code is working correctly:
    a. The current year appears in the footer on all pages

Part 5: Browser Check, Console Check and Validation

- Double check that your JavaScript file is linked correctly on all HTML pages
- Confirm the greeting update on the home page and does not create console errors for the other pages
- The footer year is dynamic and works correctly on all pages
- There are no additional console errors
- Check your HTML code by using W3C Validator: https://validator.w3.org/
- Fix major errors

Part 6: GitHub and Deployment

Add Increment 3 to GitHub

1. Select your github repository named 67250_techproject_phaseb
2. In the blue section you see on your screen click “upload an existing file” and drag and drop your andrewID-increment3 folder. Drag the full folder so it is in the root of your repo.
3. After all your files upload, click Commit changes (green button at bottom).

Optional: VS Code will also integrate with GitHub if you’d like to use those features instead.

Deploy on GitHub Pages

1. In GitHub, go to Repository → Settings → Pages
2. Select:
    - Branch: main
    - Folder: / (root)
3. Click Save

Your site will deploy from the root of your repository.

Note: Since Increment 3 lives inside andrewID-increment3, you may need to manually navigate to that folder in the deployed URL. For example, the URL will be something like: https://jeriaq.github.io/67-250_techproject_phaseb/andrewid-increment3/index.html

Important Notes

- You can create a landing page (index.html) that sits in your root
- You do NOT need to create a new repository for each increment.
- You do NOT need to create new branches for each increment.

Part 7: Submission

You do not need to upload anything to Canvas.

To receive credit:

- Your GitHub repository must be public
- The andrewID-increment3 folder must exist
- Files must be properly structured and accessible

TAs will review your GitHub repository directly.

🎉 Congratulations on completing Web Development Increment 3! 👏

Web Dev Incement 4

67250 IS MILIEUX

Phase B: Web Design, Development & Implementation
Increment 4: CSS Layouts, DOM events, jQuery and Active Nav-Bar

---

Overview

In this increment you will integrate **JavaScript**, **jQuery**, and a **basic API call** to make your museum website more interactive and user-friendly. You will implement event-driven interactions (buttons and toggles), dynamically update content, and validate user input using an API.

By the end of this increment, you will have:

- A Read More / Read Less toggle (jQuery)
- A button-triggered reveal interaction
- A basic API-powered validation or lookup that improves the ticket/checkout experience

By the end of this increment, you will be able to:

- Create interactive web features using JavaScript and jQuery (e.g., buttons, toggles like Read More / Read Less)
- Build a consistent, responsive layout using CSS Grid and Flexbox across multiple pages
- Use event-driven programming to control what users see and do (e.g., showing forms, triggering actions)
- Enhance user experience with dynamic behavior (e.g., active navigation bar, API-based validation)

Part 1: Directory Structure and File Setup

To begin Increment 4, you must have successfully completed Increment 1-3. You should also have a completed design guide and low-fidelity wireframes that demonstrate clear attention to the information architecture, which you are now using to develop your high-fidelity wireframes.

1. Inside your `67250_technologyproject_phaseb` folder, create a new folder named:

```
None
andrewID-increment4
```

2. Copy your entire increment 3 content into this new folder.
3. Confirm the following:
    - `index.html` is in the root folder
    - A `views` subfolder exists for your other pages
    - A `static` subfolder exists for assets (images, CSS, JS)

```
None
andrewID-increment3/
|
├── index.html
├── views/
|   ├── checkout.html
|   ├── exhibitions.html
|   ├── explore.html
|   └── etc...
|
├── static/
|   └── (images, etc.)
|   └── style.css
|   └── script.js
```

Part 2: Page Layout (CSS Grid)

In this part of the increment, your goal is to take what you've built so far and turn it into a clean, consistent, and well-structured website—not just individual pages.

Finish your wireframes (your plan matters)

- Make sure your wireframes are complete and reflect your intended design
- It's okay (and expected) that your wireframes look different from the examples in the increments
- Use the steps in each increment to build toward your own design, not copy ours
- The increments give you the tools—your wireframes show how you want to use them

Make all pages look and feel the same (consistency)

- Every page on your site should feel like it belongs to the same website
- This means:
    - The header should look the same on every page
    - The navigation bar should be in the same place and work the same way
    - The footer should be consistent

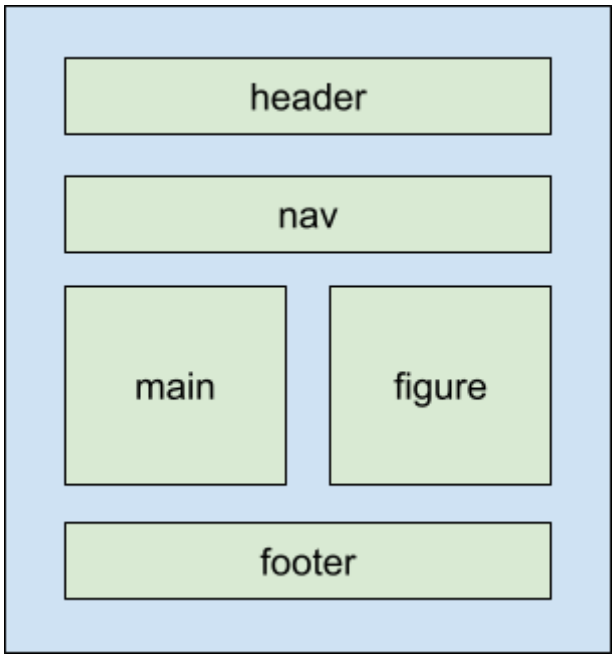
- The overall layout should not change from page to page
- A user should be able to move between pages without feeling confused

What is CSS Grid (and why are we using it)?

- What it is: CSS Grid is a layout system that lets you divide your page into rows and columns, like a table—but much more flexible.
- What it does: It allows you to place different parts of your page (header, nav, content, etc.) exactly where you want them.
- w3Schools has a great summary and code example of CSS Grid

Example idea…With Grid, you can say:

- Header goes across the top
- Navigation goes below it
- Content is split into left and right sections (like text and an image)
- Footer goes at the bottom



Why we use it:

- It makes your layout organized and predictable
- It helps keep your pages consistent across the site
- It's easier to manage than manually positioning everything
- It allows you to reuse the same layout on every page
- Instead of guessing where things go, you're defining a clear layout structure

Big picture: In this increment we are moving from “a bunch of pages with content” to “a well-designed website with structure, consistency, and layout control”

Simple way to think about it:

HTML = what your content is

CSS Grid = where everything goes

This results in consistency = making every page feel the same

Broadly, to make this happen you will need to employ a strict structural model:

- Main Container: Wrap all page content in a single top-level
- Contained Items: Organize all elements (e.g., <header>, <nav>, <main>) as direct children. Use semantic HTML5 elements instead of generic divs
- Class Naming: Assign a unique, descriptive CSS class to every item within the main container
- Style the elements using CSS Grid properties

Wrap your HTML in a main container and organize the contained elements

1. This is going to require a little bit of tedious HTML rework but it will be worth it
2. Start by adding a main container `<div class="container">` to wrap everything inside the <body> of all of your HTML documents. Don’t forget to close the new `<div>` just before the `</body>`
3. Add an attribute to `<header>` with a `class=”header”`
4. Move <nav> out of `<header>` so it is no longer a child.
5. Add an attribute to `<nav>` with a `class=”nav_bar”`
6. Change to `<main class=”main>` so it uses the semantic element `<main>`
7. Add an attribute to `<footer>` with a `class=”footer”`

The new container and structure should look something like this in *each* HTML document:

```HTML
<div class="container">
    <header class="header"></header>
    <nav class="nav_bar"></nav>
    <main class="main"></main>
    <figure class="main_figure"></figure>
    <footer class="footer"></footer>
</div>
```

8. Style the elements using CSS Grid properties. Copy and paste the following code into your style.css file:

```
CSS
.container {
  display: grid; /* Turns this div into a CSS Grid layout */
  grid-template-areas:   /* Defines the layout structure */
    'header header'
    'nav_bar nav_bar'
    'main main-img'
    'footer footer';
}

/* Places each element into the correct area of the grid */
.header {
   grid-area: header;
}
.nav_bar {
   grid-area: nav_bar;
}
.main {
  grid-area: main;
}
.main_img {
   grid-area: main-img;
}
.footer {
   grid-area: footer;
}
```

The above CSS code is enforcing the grid layout and priority.

Property	Target	Purpose
`display: grid;`		Establishes the primary layout manager It tells the browser: "I'm going to organize this page using a grid layout"
`grid-template-areas`		**Visually defines the exact structure** and placement of all named elements, guaranteeing parallel design consistency when repeated

		across pages It tells the browser: “This is where everything goes on the page”
`grid-area:`	Each element in the grid	**Names each element** for placement within the template (e.g., `header`, `main`) It tells the browser: “Put this element in this spot on the layout”

9. Verify that your website's recurring elements (such as the header navigation bar and footer) are correctly positioned and maintain their location consistency as you navigate through different pages.

Part 3: Page Layout (Flexbox Layout)

Now that you’ve used CSS Grid to structure your page layout, we’re going to introduce Flexbox, which helps you control what happens inside each section of your page.

Flexbox (Flexible Box Layout) is a CSS tool that lets you align, space, and organize content in a row or column. It’s most useful for controlling how items behave inside a container (like a navigation bar or header).

Flexbox makes it easy to:

- Center content (horizontally and vertically)
- Space items evenly (like menu links in a nav bar)
- Align elements cleanly without using hacks
- It helps your content look clean, balanced, and organized

How is Flexbox different from CSS Grid?

- CSS Grid = controls the overall page layout (big picture)
- Flexbox = controls the layout inside a section (details)

Think of it like:

- Grid = where sections go. Grid builds the structure of the page.
- Flexbox = how things are arranged inside those sections. Flexbox makes the content inside that structure look good.

You'll often use Grid and Flexbox together:

- Grid to place your header, nav, main, and footer
- Flexbox inside those sections to align content (like centering text or spacing links)

Right-align the navigation in your grid

1. Select the .nav_bar you used for the grid layout and right-align the content by adding to your current CSS code the new declarations for `display` and `justify-content`

```
CSS
.nav_bar {
   grid-area: nav_bar;
   display: flex; /* Uses Flexbox for internal layout */
   justify-content: right;   /* Right aligns the content */
}
```

2. Verify your website's navigation is correctly positioned and maintains it location consistency as you navigate through different pages

Part 4: Active Navigation Bar

Recall that in Increment 2, you used a unique ID (e.g., `<a id="navselected">`) to manually highlight the current page in the navigation bar.

In this increment, you will remove those hard-coded IDs and replace them with a dynamic "active navigation" feature that automatically highlights the correct page based on the URL.

1. Remove the `id="navselected"` attribute in each of your HTML files
2. In `style.css` replace: `#navselected` with `.active`
3. Be sure to define a style for the `.active` class selector to make the current page's navigation link visually prominent
4. In `script.js` implement a function that executes upon every page load. This function should:
   - Select all navigation links within the navbar.
   - Identify the currently loaded page using `window.location`.
   - Apply a distinct styling class, "active," to the link corresponding to the current page.
5. Verify your website's navigation is correctly functioning (and the color is changing on the selected page) as you navigate through different pages

Code Example:

```javascript
/* Sets the 'active' class on the navigation link that matches the current page URL.*/

function ActiveNav() {
    // Get all navigation links
    const navLinks = document.querySelectorAll('nav li a');

    // Iterate over each link
    navLinks.forEach(link => {
    // Check if the link's href matches the current window location
        if (window.location.href === link.href) {
            // Add the 'active' class to highlight the current page
            link.classList.add("active");
        }
    });
}

// Execute the function to set the active navigation link on page load
ActiveNav();
```

Part 5: “Read More / Read Less” Toggle Using jQuery (index.html)

In this section of the increment you will work with jQuery to reduce clutter while still providing information on the homepage.

Add the jQuery Library to Your Increment

1. Add the jQuery library to the `<head>` of index.html. Refer to the following [JQuery CDN]() site, select the appropriate JQuery release (jQuery 4.0.0), and copy and paste over the corresponding script (minified).
2. Note: jQuery only needs to be included on pages where it is used (index.html for this increment). If you attempt to use jQuery ($) on a page where the library is not included, it will result in an error.

Add short + long content sections to index.html

1. On `index.html`, under your main introduction and buy tickets button, add:
    - A short paragraph shown by default
    - A longer paragraph hidden by default
    - Button for Read More shown by default
    - Button for Read Less hidden by default
2. Give each <p> and a unique ID (such as id=”shortIntro” “readMore”, etc
3. Ensure the text and buttons are properly styled using CSS in `style.css`. You will need to hide the long text and Read Less button with the `display: none` deceleration.

Use jQuery to Toggle Content

1. Open `script.js` and add to new jQuery functions:
    a. Clicking Read More:
        - Shows the long paragraph
        - Hides the Read More button
    b. Clicking Read Less:
        - Hides the long paragraph
        - Shows the Read More button again
2. Check that the toggle works correctly without refreshing the page.

Code Example:

```javascript
 // When the "Read Less" button is clicked
 $("#readLess").click(function(){
    $("#longIntro").hide(); // Hide the long introduction text
    $("#readLess").hide();  // Hide the "Read Less" button itself
    $("#readMore").show();  // Show the "Read More" button

  });

// When the "Read More" button is clicked
  $("#readMore").click(function(){
    $("#longIntro").show();  // Show the long introduction text
    $("#readLess").show();   // Show the "Read Less" button
    $("#readMore").hide();   // Hide the "Read More" button
  });
```

Code Explanation: This code waits for the user to click either the “Read More” or “Read Less” button. When clicked, it shows or hides the longer text and switches which button is visible. This allows users to expand and collapse content without reloading the page, making the website more interactive and easier to navigate.

Part 6: Button-Based Interaction: Improve the Ticket Buying Experience

In this part of the increment, your goal is to make your museum website more interactive and easier to use by adding a button-based interaction with JavaScript.

Rather than sending users across multiple separate pages, you will improve the ticket-buying experience by creating a single page where visitors can:

- View ticket prices
- Browse available dates

- Click a Buy Now button
- Reveal a ticket purchase form

This activity helps you practice event-driven web design, where something happens on the page when a user clicks a button. It also helps you connect HTML, CSS, and JavaScript in a way that improves the user experience.

You will create a buytickets.html page that includes:

- Ticket pricing information,
- A monthly ticket schedule displayed in a table,
- A Buy Now button for each available date,
- A purchase form that is hidden by default,
- A submit button that displays the alert: "redirecting to payment system."

The purchase form should only appear after a user clicks one of the Buy Now buttons.

Create or update buytickets.html

1. Create a page called buytickets.html (or update your existing tickets page so it serves this purpose). This page should bring together both:
    - ticket information (tickets.html)
    - and the checkout interaction (checkout.html)
2. Add ticket prices and a monthly schedule. On buytickets.html, use HTML to show ticket prices and a monthly schedule of available dates. Use at least one <table> element to organize this information clearly. For each available ticket date, include a Buy Now button.
3. Add a purchase form. Below the table, create a form users can use to purchase tickets. For example, your form might include fields such as:
    - Name
    - Email
    - Number of tickets
    - Selected date
    - Inside the purchase form, include a submit button.

Hide the form by default with CSS in `style.css`

4. This form should be present in the HTML, but it should not be visible when the page first loads. Use CSS to hide the purchase form when the page first opens.

Add a JavaScript function in script.js

5. In your script.js file, write a function that reveals the hidden purchase form.
6. Then connect each Buy Now button to that function using an onclick event.
7. When the user clicks a Buy Now button:
    - The form should appear
    - The page should feel interactive without needing to reload

8. Add a submit interaction. Attach an onclick event so that when the user clicks the submit button, an alert appears with this exact message: "Redirecting to payment system." This simulates the next step in an online checkout process.

Test the Full Interaction

9. Before moving on, make sure your page works as expected:
    - The ticket form is hidden when the page loads
    - Clicking Buy Now reveals the form
    - The interaction is handled by a JavaScript function in script.js
    - Clicking the submit button shows the alert
    - The page is easy to understand and use

Part 7: Browser Check, Console Check and Validation

Verify that all the following items have been completed:

- Page Layout Consistency: The page layout is uniform throughout the site.
- "Read More" Functionality: The jQuery functions for "Read More" are operating as expected.
- Buy Tickets Page: The design is effective, and the functionality is correct.
- Navigation: The navigation bar accurately highlights the current page.
- Fix any console errors
- Check your HTML code by using W3C Validator: https://validator.w3.org/
- Fix major errors

Part 8: GitHub and Deployment

Add Increment 4 to GitHub

1. Select your github repository named 67250_techproject_phaseb
2. In the blue section you see on your screen click “upload an existing file” and drag and drop your andrewID-increment4 folder. Drag the full folder so it is in the root of your repo.
3. After all your files upload, click Commit changes (green button at bottom).

Optional: VS Code will also integrate with GitHub if you’d like to use those features instead.

Deploy on GitHub Pages

1. In GitHub, go to Repository → Settings → Pages
2. Select:
    - Branch: main
    - Folder: / (root)
3. Click Save

Your site will deploy from the root of your repository.

Note: Since Increment 4 lives inside andrewID-increment4, you may need to manually navigate to that folder in the deployed URL. For example, the URL will be something like: https://jeriaq.github.io/67-250_techproject_phaseb/andrewid-increment4/index.html

Important Notes

- You can create a landing page (index.html) that sits in your root
- You do NOT need to create a new repository for each increment.
- You do NOT need to create new branches for each increment.

Part 9: Submission

You do not need to upload anything to Canvas.

To receive credit:

- Your GitHub repository must be public
- The andrewID-increment4 folder must exist
- Files must be properly structured and accessible

TAs will review your GitHub repository directly.

🎉 Congratulations on completing Web Development Increment 4! 👏

Web Dev Incement 5

67250 IS MILIEUX

Phase B: Web Design, Development & Implementation
Increment 5: RWD, Embedded Media, APIs + Testing

Overview

For this increment, you are tasked with enhancing your museum website with responsive web design, to include a mobile layout. You must also integrate external content, specifically a YouTube video, and incorporate at least one JavaScript API such as Lealeit Maps to add interactivity. Finally, you will complete a minimal testing checklist to verify the site's functionality across all pages and screen sizes.

Part 1: Directory Structure and File Setup

1. Create a new folder: `andrewID-increment5` inside your `67250\_webdevincrements` folder.
2. Copy your complete Increment 4 website into this new folder.
3. Verify the folder structure:

```
None
andrewID-increment5/
├── index.html
├── views/
│   ├── tickets.html
│   ├── exhibitions.html
│   └── etc...
└── static/
    ├── style.css
    └── script.js
```

Part 2: Responsive Web Design

Extend your existing grid layout with a dedicated mobile design.

1. Mobile Wireframe: Sketch a design for the mobile version.
2. Viewport Meta Tag: Include (or confirm you already have) the following in your HTML `<head>` for proper mobile rendering:

```html
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

3. Media Query: Set up a media query in style.css to apply mobile-specific styles.

```css
@media not|only mediatype and (expressions) {

   /* Your CSS code for the mobile layout */

}
```

4. Mobile Layout: Define and apply new CSS rules within the media query for the mobile layout.

Part 3: Responsive Navigation Bar

Implement a navigation bar that is responsive: hide regular links on small screens and show a hamburger icon instead.

- **HTML:** Add a link or button for the hamburger icon inside the navbar.
- **CSS** (`style.css`):
  - Hide the hamburger icon by default.
  - Create a media query for screens smaller than 650px.
  - Inside the media query, hide the regular navigation links and display the hamburger icon (you may keep the first link visible).
- **JavaScript** (`script.js`)**:** Create a toggle function for the hamburger icon. When clicked, this function should add or remove a responsive class from the navbar links to control their visibility on small screens.

Part 4: Embed a YouTube Video (`<iframe>`)

Select a page on your website to embed a museum-related YouTube video using an `<iframe>`. If a YouTube video is unavailable, or you cannot find a suitable YouTube video, you have the option to embed an alternative HTML page or element within the iframe instead.

1. Go to YouTube and choose a museum-related video (art, history, exhibits, education, etc.)

2. Click Share → Embed, and copy the `<iframe>` code
3. Paste the `<iframe>` into your HTML file
4. Adjust the `<iframe>` size to fit your layout. Remove the default `width` and `height` attributes from the `<iframe>` tag so CSS can control its size
5. Confirm the video loads and plays. If the video shows “video unavailable,” it is likely copyright protected - choose a different video instead

Important: Embedded content often requires an HTTP server. If you see errors like "Video configuration error" or "unavailable" when viewing locally, use the following method:

1. Open terminal
2. Navigate into your project folder (the one with your files)
3. From inside your increment folder, run: `python3 -m http.server 8000`
4. Access your site via a URL like: `http://localhost:8000/views/the_relevant_page.html`

Part 5: API Integration OpenStreetMap + Leaflet

Leaflet is an open-source JavaScript library for creating interactive, mobile-friendly maps. You are going to replace your Google Map `<iframe>` code with the Leaflet API.

Setup Requirements

Begin by reviewing the Quick Start Guide. To implement Leaflet, the following steps are necessary:

1. Include the required Leaflet CSS and JavaScript files
2. Add a `<div>` element to serve as the map container
3. Ensure the map container has a defined, fixed height

Creating the Map

The map is initialized and displayed in two main steps in your script.js:

4. Initialize the Map: Use `L.map()` to create the map instance, setting the initial coordinates (`lat`, `lng`) and zoom level (`zoom`).

Creating the Map

The map is initialized and displayed in two main steps in your script.js:

5. Initialize the Map: Use `L.map()` to create the map instance, setting the initial coordinates (`lat`, `lng`) and zoom level (`zoom`).

```javascript
var map = L.map('map').setView([lat, lng], zoom);
```

6. Add a Tile Layer: A tile layer, such as one from OpenStreetMap, must be added to the map to display the visual geography.

```javascript
L.tileLayer(...).addTo(map);
```

Adding Elements (Example):

- Add a marker to the map: `L.marker([lat, lng]).addTo(map);`

Notes:

- If your map does not appear, check that you are not declaring `var l` in the current page URL code from Increment 3. If so, change `l` to `currentpage`
- If you receive console errors such as '$ is not defined', make sure you include jquery in your header.
- Check the site using `python3 -m http.server 8000`.
- If you are still unable to see the map on visit.html, consider removing integrity attributes when loading Leaflet.

Part 6: CHALLENGE (Optional) Extra API Integration

You are required to integrate at least one API in your Phase B project. (the Leaflet map API satisfies this requirement).

For an extra challenge, try one of the following:

- Replace the YouTube iframe with the YouTube API
- Replace the Google Maps iframe with the Google Maps API
- Display weather data in the footer using a public weather API
- Add advice using Advice Slip

Part 7: Browser Check, Console Check and Validation

Verify that all the following items have been completed:

- **Navigation Test:** Click every navigation link on every page (no broken links or 404s)

- YouTube video loads and plays
- The map with Leaflet is visible and correct
- Flexbox layout works and looks organized
- Check your HTML code by using W3C Validator: https://validator.w3.org/
- Fix major errors

Part 8: Function Invocation & Console Errors

Confirm:

- Console Test: Open DevTools → Console on each page (no red errors)
- Functions only run on pages where required elements exist
- No `TypeError` or `ReferenceError` messages appear in the console

If needed:

- Check `window.location.href`
- OR confirm elements exist before updating them

Part 9: GitHub and Deployment

Add Increment 5 to GitHub

1. Select your github repository named 67250_techproject_phaseb
2. In the blue section you see on your screen click "upload an existing file" and drag and drop your andrewID-increment5 folder. Drag the full folder so it is in the root of your repo.
3. After all your files upload, click Commit changes (green button at bottom).

Optional: VS Code will also integrate with GitHub if you'd like to use those features instead.

Deploy on GitHub Pages

1. In GitHub, go to Repository → Settings → Pages
2. Select:
    - Branch: main
    - Folder: / (root)
3. Click Save

Your site will deploy from the root of your repository.

Note: Since Increment 5 lives inside andrewID-increment5, you may need to manually navigate to that folder in the deployed URL. For example, the URL will be something like: https://jeriaq.github.io/67-250_techproject_phaseb/andrewid-increment5/index.html

Important Notes

- You can create a landing page (index.html) that sits in your root
- You do NOT need to create a new repository for each increment.
- You do NOT need to create new branches for each increment.

Part 10: Submission

You do not need to upload anything to Canvas.

To receive credit:

- Your GitHub repository must be public
- The andrewID-increment5 folder must exist
- Files must be properly structured and accessible

TAs will review your GitHub repository directly.

🎉 Congratulations on completing Web Development Increment 5! 👏